AI in the Sciences and Engineering HS2024

Moyang Li

January 21, 2025

Contents

1 Training the FNO to Solve the 1D Wave Equation **3**

1.1 Task 1: One-to-One Training . . . 3

1.2 Task 2: Testing on Different Resolutions . . . 3

1.3 Task 3: Testing on Out-of-Distribution (OOD) Dataset . . . 3

1.4 Task 4: All2All Training . . . 4

2 Bonus Task **5**

3 PDE-Find: Reconstructing PDEs from data **6**

3.1 Data Visualization and Guess of PDEs . . . 6

3.2 Algorithm Implementation and Experiments . . . 6

4 Foundation Models for Phase-Field Dynamics **8**

4.1 Data Generation . . . 8

4.2 Time-dependent Neural Solver Development . . . 8

4.3 Experiments . . . 8

5 Bonus Task: Stability Analysis **10**

5.1 Choice of Test Function . . . 10

5.2 Handling the Nonlinear Term $f(u)$ . . . 10

5.3 Energy Estimate and Grönwall's Inequality . . . 10

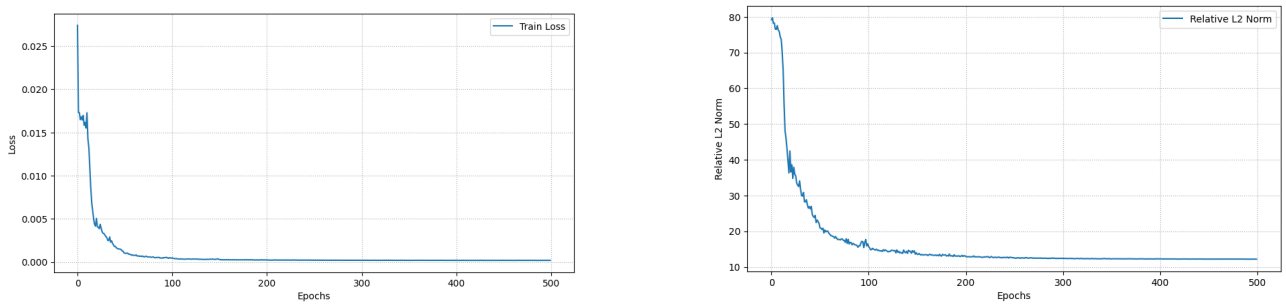
5.4 Dependence on ϵ . . . 11

5.5 Conclusion . . . 11

1 Training the FNO to Solve the 1D Wave Equation

1.1 Task 1: One-to-One Training

- Prepare the training dataset through choosing trajectories at $t = 0.0$ for the input and the trajectories at $t = 1.0$ as the output, create the grid by calling torch.linspace(0, 1, data_dim) and add the grid to the input data.
- Call np.random.choice to randomly sample 64 trajectories for training and use the remaining 64 trajectories for validation.
- Adjust the training parameters by setting the epoch as 500 to ensure convergence. Fig. 1 demonstrates that the model has converged.
- The final train loss in the training set is 0.0001976 and the average relative error L2 in the validation set is 12.7562 % when epoch = 500.
- Test the trained model on test_sol.npy, the final average relative L2 error is 13.9167 %.



(a) Training Loss

(b) Relative L2 Error on Validation Sets

Figure 1: Visualization of Convergence and Validation Error

Dataset	Validation Set	test_sol.py	OOD
Average Relative L2 Error (%)	12.7562	13.9167	17.9192

Table 1: Comparisons of Validation Set, `test_sol.py` and OOD datasets for One-to-One Training.

1.2 Task 2: Testing on Different Resolutions

Resolution	32	64	96	128
Average Relative L2 Error (%)	14.4554	13.8945	13.8042	13.8601

Table 2: Average Relative L2 Error on Datasets with Different Resolutions.

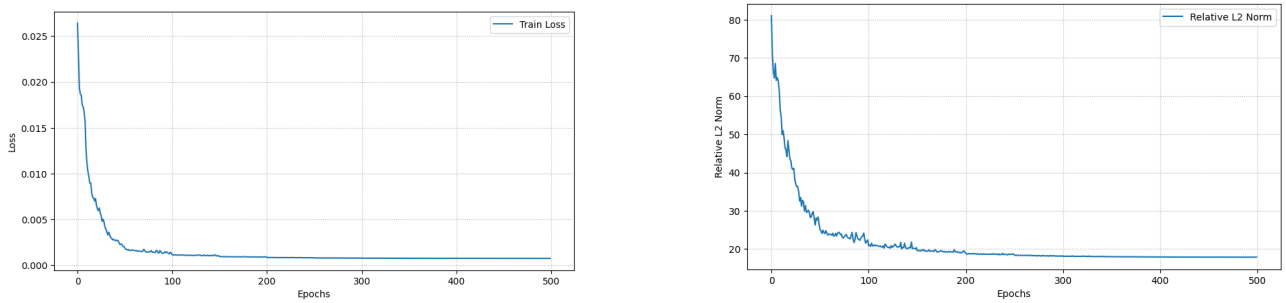
Table 2 shows the relative L2 error on datasets with different resolution. From the table, we can observe that higher resolution usually leads to higher accuracy as high-resolution data can provide more frequency information. However, the average relative L2 error does not always decrease with the increase of resolution if the resolution is large enough to provide sufficient information.

1.3 Task 3: Testing on Out-of-Distribution (OOD) Dataset

The relative L2 error on OOD dataset is 17.9192 % as shown in Table 1, which is higher than the error observed in task 1 due to the OOD. This result is reasonable as there exists large domain shift between training and testing datasets.

1.4 Task 4: All2All Training

For the All2All Training, I only change model as time-dependent FNO and construct new dataset suitable for this task. I keep other hyper-parameters of training the same with Task 1. Fig. 2 demonstrates the convergence of time-dependent FNO model.



(a) Training Loss

(b) Relative L2 Error on Validation Sets

Figure 2: Visualization of Convergence and Validation Error for All2All Training.

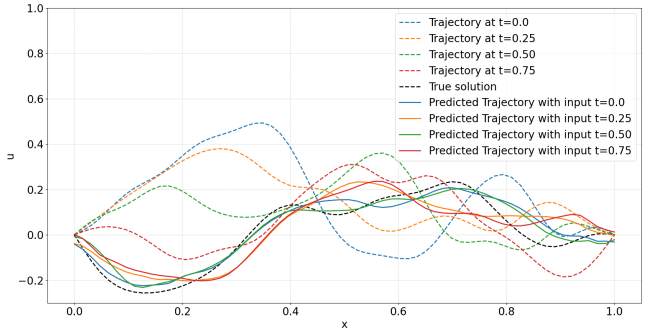


Figure 3: Prediction at $t = 1.00$ with Different Input Time Steps.

t_{input} \ t_{predict}	0.25	0.50	0.75	1.00
0.00	6.3915	10.7202	15.2340	22.1698
0.25	-	20.5983	14.7938	28.7019
0.50	-	-	14.0035	21.1539
0.75	-	-	-	24.3494
Avg.	6.3915	15.6592	14.6771	24.0938

Table 3: Average Relative L2 Error on Test Dataset for Different Input Time and Prediction Time.

I use randomly sampled 64 trajectories from Task 1 as training data (ensuring the same training data and fair comparisons) and create $5 \times 4/2 = 10$ time pairs for each trajectory. Therefore, we have 640 samples for training in total. I report the average relative L2 error on test sol.npy dataset in the last column of Table 3, which demonstrates the larger prediction error (24.0938%) compared with Task 1 (13.9167%). The possible explanation is that capturing time dependency causes difficulty for the training of time-dependent FNO model. Fig. 3 shows that the model can capture the overall trend of trajectory but not accurate enough.

2 Bonus Task

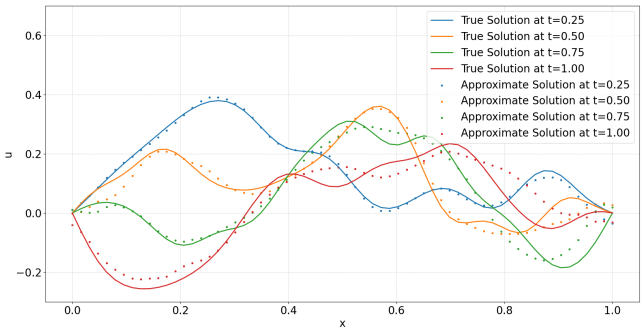


Figure 4: Prediction at Different Steps with Same Input Time Steps.

Dataset	Validation Set	test_sol.py	OOD
Average Relative L2 Error (%)	17.8950	17.8116	27.1950

Table 4: Comparisons of Validation Set, `test_sol.py` and OOD datasets for All2All Training.

Table 3 presents the prediction error at different time steps with various input time steps. Fig. 4 shows one visualization of a trajectory prediction at different time steps ($t = 0.25, t = 0.50, t = 0.75, t = 1.00$) with $t_{\text{input}} = 0.00$.

From Table 3, we can observe that short-time prediction is easier than long-time prediction. Given the example of prediction with input timestamp at $t = 0.00$ (first row in Table 3), larger prediction interval [1] will lead to higher average relative L2 error. However, from Fig. 4, we can see that the implemented time-dependent FNO model successfully captures the trend of the trajectory in different time steps even if the accuracy is lower for long-time prediction. Furthermore, the accuracy of the model decreases with increasing prediction time steps, as shown in last row of the Table 3, due to more long-time prediction at later prediction time steps. For example, the prediction at $t = 1.00$ includes $\{[u(t = 0.00), u(t = 0.25), u(t = 0.50), u(t = 0.75)] \rightarrow u(t = 1.00)\}$.

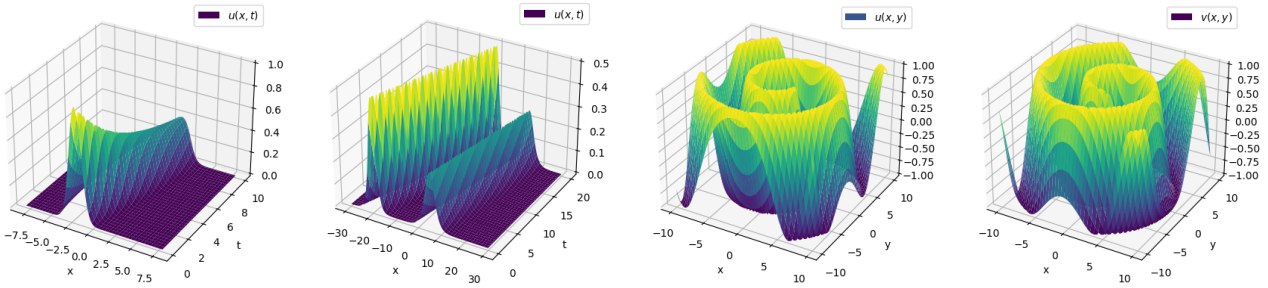
In addition, Table 3 also show that the prediction with earlier input time steps is easier than others. For diagonal and subdiagonals of Table 3, I compare the average relative L2 error with the same prediction interval but different input time steps. We can find that the accuracy of the prediction at $t_{\text{input}} = 0.00$ is higher than others. I initially think the possible reason is that "All2All Training" is an imbalanced problem as the data with $t_{\text{input}} = 0.00$ occupies higher proportion. Imbalancement will cause the model to favor the majority class ($t_{\text{input}} = 0.00$). Thus, I construct a balanced dataset which includes time pairs $\{(0.00, 0.25), (0.25, 0.50), (0.50, 0.75), (0.75, 1.00)\}$. However, the experiments on balanced datasets also show this phenomenon (please look at the section `Additional Experiment: Balanced Training` in `FNO_wave_equation/ solution.ipynb` for more details).

From Table 4, we can see that the OOD problem also causes performance degradation, which is similar to Task 3.

[1] prediction interval: the time difference between the input and prediction time steps.

3 PDE-Find: Reconstructing PDEs from data

3.1 Data Visualization and Guess of PDEs



(a) $u(x,t)$ of File 1. (b) $u(x,t)$ of File 2. (c) $u(x,y)$ of File 3. (d) $v(x,y)$ of File 3.

Figure 5: Data Visualization of 3 Files.

Fig. 5 presents the visualization of 3 files. As shown in Fig. 5a the data distribution of file `"1.npz"` is similar to the Burgers equation, the form of which is $u_t + uu_x - \epsilon u_{xx} = 0$. Fig. 5b demonstrates that the file 2 `"2.npz"` is the KdV equation, whose form is $u_t + 6uu_x + u_{xxx} = 0$. From Fig. 5c and Fig. 5d, we can know that the file3 `"3.npz"` comes from Reaction Diffusion equation and the form of this equation is shown as following:

$$u_t = 0.1\nabla^2 u + \lambda(A)u - \omega(A)v \tag{1}$$

$$v_t = 0.1\nabla^2 v + \omega(A)u + \lambda(A)v \tag{2}$$

$$A^2 = u^2 + v^2, \omega = -\beta A^2, \lambda = 1 - A^2 \tag{3}$$

3.2 Algorithm Implementation and Experiments

For the PDE-FIND of `"1.npz"`, I follow the guidance of PDE-FIND[2] [3] and first construct following candidate term library with finite difference:

$$u_t = \mathcal{D}(1, u, u^2, u^3, u_x, uu_x, u^2u_x, u^3u_x, u_{xx}, uu_{xx}, u^2u_{xx}, u^3u_{xx}, u_{xxx}, uu_{xxx}, u^2u_{xxx}, u^3u_{xxx}). \tag{4}$$

Then I perform Ridge Regression to find the corresponding PDE equation. I get the PDE equation for file 1 of:

$$u_t = (-1.000403 + 0.000000i)uu_x + (0.100145 + 0.000000i)u_{xx}. \tag{5}$$

The approximate form of this solution is

$$u_t = -uu_x + 0.1u_{xx}, \tag{6}$$

which is consistent with my hypothesis.

For the PDE-FIND of `"2.npz"`, I first call `build_linear_system(u, dt, dx, D=3, P=2)` to con-

[2]Rudy, Samuel H., et al. "Data-driven discovery of partial differential equations." Science advances 3.4 (2017): e1602614.

[3]https://github.com/snagcliffs/PDE-FIND.git.

struct following candidate term library with finite difference:

$$u_t = \mathcal{D}(1, u, u^2, u_x, uu_x, u^2u_x, u_{xx}, uu_{xx}, u^2u_{xx}, u_{xxx}, uu_{xxx}, u^2u_{xxx}). \tag{7}$$

Here I set the maximal degree of nonlinear term as 2 and degree of partial derivative as 3. And the choice of degree in this task is essential.

Then I call `TrainSTRidge(Theta,Ut,10**-5,5)` to solve it and get following equation:

$$u_t = (-5.955400 + 0.000000i)uu_x + (-0.987754 + 0.000000i)u_{xxx}, \tag{8}$$

which can be approximated as following:

$$u_t = -6uu_x - u_{xxx}. \tag{9}$$

This form is consistent with my previous assumption. To verify the effectiveness of size $\mathcal{D}$, I choose the same library as shown in Eq. 4 for `2.npz` and get following wrong solution:

$$u_t = (-7.255936 + 0.000000i)u^2u_x. \tag{10}$$

This experiment demonstrates that too large the library size will lead to inaccurate prediction.

For the PDE-FIND of `"3.npz"`, I first randomly sample 5000 points for the coordinates xy and choose 30 time steps for t corresponding to every coordinate to reduce the solving time. Then I construct $\Theta(U)$ and compute U_t. I call `PolyDiffPoint()` to compute $\mathcal{D}_1 = (1, u_x, u_y, u_{xx}, u_{xy}, u_{yy}, v_x, v_y, v_{xx}, v_{xy}, v_{yy})$ and u_t, v_t, then I call `build_Theta()` to build a 3-degree library. The size of library is:

$$N(\mathcal{D}_1) \times N(\mathcal{D}_2) = 11 \times 10 = 110, \tag{11}$$

where $\mathcal{D}_2 = (1, u, v, u^2, uv, v^2, u^3, u^2v, uv^2, v^3)$. to build the library.

Finally, I call `TrainSTRidge()` to find the solution for u_t, v_t respectively:

$$u_t = 0.099972u_{xx} + 0.100004u_{yy} + 0.999913u + 0.999979v^3 - 0.999882uv^2 + 0.999977u^2v - 0.999900u^3, \tag{12}$$

$$v_t = 0.100032v_{xx} + 0.099968v_{yy} + 1.000278v - 1.000293v^3 - 0.999971uv^2 - 1.000268u^2v - 0.999978u^3, \tag{13}$$

which can be approximated as following equations:

$$u_t = 0.1\nabla^2 u + \lambda(A)u - \omega(A)v, \tag{14}$$

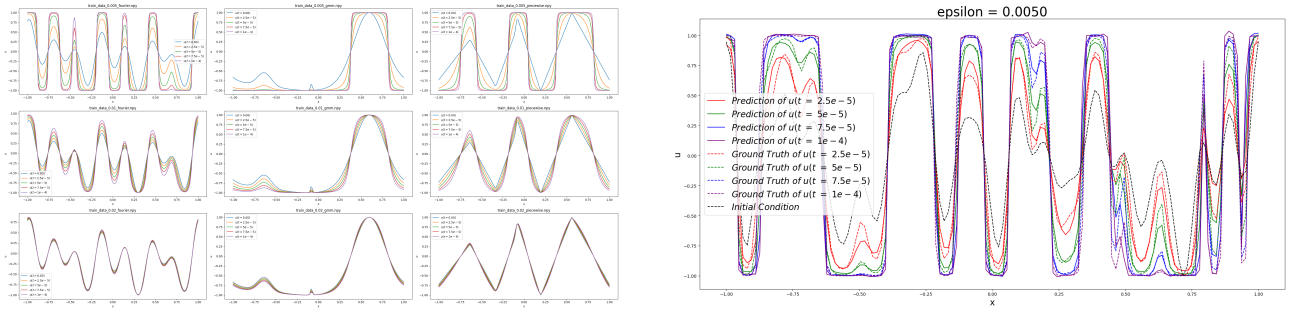
$$v_t = 0.1\nabla^2 v + \omega(A)u + \lambda(A)v, \tag{15}$$

$$A^2 = u^2 + v^2, \omega = -A^2, \lambda = 1 - A^2. \tag{16}$$

Through the experiments on 3 files, we can find that PDE-FIND works well for constant-coefficient PDE. However, PDE-FIND does not consider the situation of variant-coefficient PDE. I think one possible direction to improve the generality is to make it suitable for variant-coefficient. We can further train a neural operator to learn the change of PDE coefficients.

4 Foundation Models for Phase-Field Dynamics

4.1 Data Generation



(a) Visualization of Training Dataset.

(b) Visualization of Dataset with Higher Frequency.

Figure 6: Visualization of Generated Data for Different Distributions and Different Epsilons.

Fig. 6 shows generated data with different types of initial condition and different ϵ. For more visualization results, please refer to my submitted file `Foundational_Models_for_Allen_Cahn/vis.ipynb`.

4.2 Time-dependent Neural Solver Development

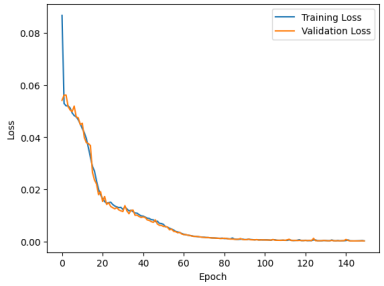


Figure 7: Training and Validations Loss Visualization.

I employ 64-dimensional positional encoding to embed the time and utilize a linear layer `nn.Linear(1, 64)` to embed ϵ. Then all these parameters will be combined with the output features before the last two layers of `FNOBlock`. Therefore, I will get the 132-dimensional features and feed this feature to the last two layers `[nn.Linear(64 * 3, 32), nn.Linear(32, 1)]` to get the final prediction.

I randomly use 7-9 modes/components/pieces to generate the training data with various initial conditions drawn from three distributions: random Fourier series, Gaussian mixtures, and piecewise linear functions. I choose 3 different ϵ values $(0.02, 0.01, 0.005)$ for the training data. I generate 1000 samples for every combination of ϵ value and the data distribution of the initial condition. Thus, the total number of training data is 9000. During training, I split the 90% training dataset for training and use the last 10% for validation. I set the number of epochs as 150 and use MSE loss to supervise the training of the model. Fig. 7 shows the good convergence of my implemented neural operator.

4.3 Experiments

ϵ	0.02	0.015	0.01	0.0075	0.005
Average Relative L2 Error (Validation) (%)	1.7562	-	2.7434	-	2.0391
Average Relative L2 Error (Testing) (%)	2.5838	8.8993	2.8863	5.7988	2.4352

Table 5: Average Relative L2 Error on Validation and Testing Datasets for Various ϵ.

Table 5 and Fig. 8a present the results on the testing datasets. I generate 200 samples for every subset of testing datasets corresponding to the specific distribution of the initial condition and the value of ϵ. In general, there are no obvious performance differences in the validation and testing datasets. Moreover, I compare the results for different ϵ, consisting of ϵ used in the training dataset $(0.02, 0.01, 0.005)$ and other ϵ not observed in the training dataset $(0.015, 0.0075)$. Table 5 shows that the neural solver implemented achieves better performance for ϵ observed in training dataset. However, the performance degradation of the neural solver in unobserved ϵ is acceptable as shown in Fig. 8b and Table 5, because two new ϵ can be obtained by interpolation of observed data.

ϵ	0.003	0.002	0.001
Average Relative L2 Error (%)	16.6671	23.0683	28.2981

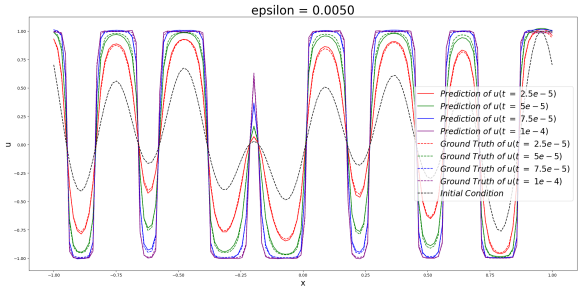
Table 6: Average Relative L2 Error on OOD Datasets with Smaller ϵ Values (Sharper Transitions).

ϵ	0.02	0.01	0.005
Average Relative L2 Error (%)	9.6484	8.9402	10.8020

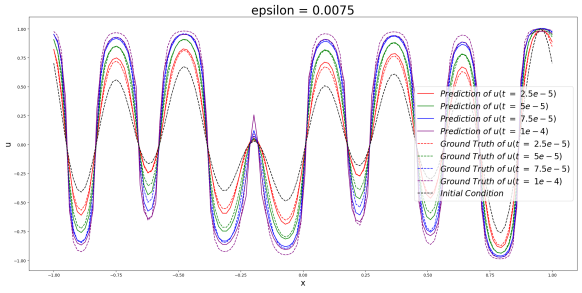
Table 7: Average Relative L2 Error on High-frequency Datasets for Various ϵ.

Besides, I also generate additional 6 datasets to evaluate the performance with higher frequency components and sharper transitions than seen in training. Table 6 and Fig. 8c show that the fewer ϵ, the performance of the model is worse. For the evaluation with high-frequency initial conditions, 25 modes/components/pieces are used to generate testing data. Table 7 and Fig. 8d show that high-frequency data will lead to performance degradation.

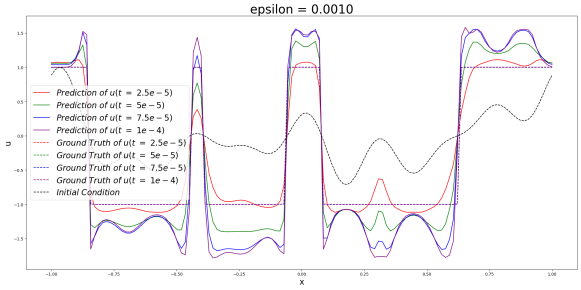
Please refer to my submitted file `Foundational_Models_for_Allen_Cahn/solution.ipynb` for more plots comparing predictions with true solutions.



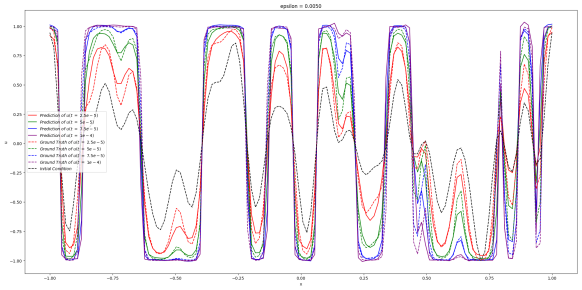
(a) Prediction on Normal Testing Dataset.



(b) Prediction with ϵ Value Interpolation.



(c) Prediction with Sharper Transitions.



(d) Prediction with High-Frequency Components.

Figure 8: Visualization of Generated Data for Different Distributions and Different Epsilons.

5 Bonus Task: Stability Analysis

I aim to make stability analysis for the Allen-Cahn equation. The analysis is shown as follows:

5.1 Choice of Test Function

Suppose $w = u - \tilde{u}$ denote the difference between the two solutions, and let $v = w$ to be the test function. Substituting w into the weak form of the Allen-Cahn equation The weak formulation of the Allen-Cahn equation can be transformed as:

$$\langle \partial_t w, w \rangle + \|\nabla w\|^2 = -\epsilon^{-2} \langle f(u) - f(\tilde{u}), w \rangle,$$

where $\langle \cdot, \cdot \rangle$ represents the L^2-inner product.

5.2 Handling the Nonlinear Term $f(u)$

Using the Lipschitz continuity of f, I linearize the term $f(u) - f(\tilde{u})$:

$$f(u) - f(\tilde{u}) = f'(\xi)(u - \tilde{u}), \quad \text{where} \quad \xi \in [u, \tilde{u}].$$

Thus,

$$|f(u) - f(\tilde{u})| \leq c_f |u - \tilde{u}| = c_f |w|,$$

where $c_f = \sup |f'(\xi)|$ is the Lipschaitz parameter.

Then, bound this into the weak formulation, the contribution of the nonlinear term becomes:

$$-\epsilon^{-2} \langle f(u) - f(\tilde{u}), w \rangle \leq \epsilon^{-2} c_f \|w\|^2.$$

Thus, it is concluded that there are nonlinear terms on the magnitude of w

5.3 Energy Estimate and Grönwall's Inequality

We bring the formula in 5.2 back to the weak formula of the Allen-Cahn equation:

$$\frac{1}{2} \frac{d}{dt} \|w\|^2 + \|\nabla w\|^2 \leq \epsilon^{-2} c_f \|w\|^2.$$

Rearranging, this becomes:

$$\frac{d}{dt} \|w\|^2 \leq 2\epsilon^{-2} c_f \|w\|^2.$$

Suppose

$$\phi(t) = \|w\|^2,$$

applying Grönwall's inequality, we can obtain following equation:

$$\phi(t) \leq \phi(0) \cdot \exp\left(2 c_f \epsilon^{-2} t\right).$$

Thus, we have:

$$\|w(t)\|^2 \leq \|w(0)\|^2 \exp\left(2 c_f \epsilon^{-2} t\right).$$

Integrating over t further provides control over $\int_0^T \|\nabla w\|^2 \, dt$ in terms of the exponential factor.

Deriving from time:

$$\int_0^T \|\nabla w\|^2 \, dt \leq \|w(0)\|^2 \exp\left(2c_f \epsilon^{-2} T\right).$$

5.4 Dependence on ϵ

From the energy inequality, we conclude:

$$\sup_{t\in[0,T]} \|w(t)\|^2 + \int_0^T \|\nabla w\|^2 \, dt \leq 2\|w(0)\|^2 \exp\left((1 + 2c_f \epsilon^{-2})T\right).$$

This shows the explicit dependence of the stability result on ϵ. The term ϵ^{-2} directly affects the exponential growth factor and bounds on the nonlinear term.

From the above, it can be concluded that the stability estimation is exponentially dependent on ϵ, so when ϵ is small, the solution is greatly affected by the nonlinear term and the error is large. This can also be verified by the physical background of the Allen-Cahn equation, i.e. when the binary alloy is separated at a certain temperature, it has energy dissipation.

5.5 Conclusion

The proof is completed by:

- Choosing $v = w$ as the test function,
- Bounding the nonlinear term using the Lipschitz property of f,
- Applying Grönwall's inequality to control $\|w(t)\|^2$ and $\|\nabla w\|^2$,
- Tracking the dependence on ϵ throughout the derivation.